Internet of Things

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Outline

- ML and Analytics in IoT
- The Strategic Value of IoT Data
- The Data Value Hierarchy
- Unique Characteristics of IoT Data
  - Time-Series Nature
  - High Velocity and Volume
  - Noise and Sparsity
- Practice code anomaly detection

ML and Analytics in IoT

- IoT in Data Value Chain, Anomaly Detection, Predictive Maintenance, and Pattern Recognition.

ML and Analytics in IoT

- The true value of IoT is not in the "Thing," but in the **insights** derived from the data those things produce.

- The Data Value Hierarchy: We move from Descriptive (What happened?) to Diagnostic (Why did it happen?) to Predictive (What will happen?) and finally Prescriptive (How can we make it happen?).

The Strategic Value of IoT Data

- Unique Characteristics of IoT Data:
- Time-Series Nature: Almost all IoT data is stamped with a temporal coordinate.
- High Velocity and Volume: Sensors can fire every millisecond, creating massive datasets that require efficient preprocessing.
- Noise and Sparsity: IoT data is often "dirty" due to sensor degradation or network jitter.

The "Edge vs. Cloud" Trade-off

- Edge: Low latency, privacy-preserving, immediate action.
- Cloud: Long-term storage, heavy-duty model training, global pattern recognition.

Anomaly Detection in IoT

- Anomaly detection is the "first line of defense" in IoT, identifying data points that do not conform to expected behavior.

- Identifying cybersecurity breaches, sensor malfunctions, or sudden industrial equipment failures

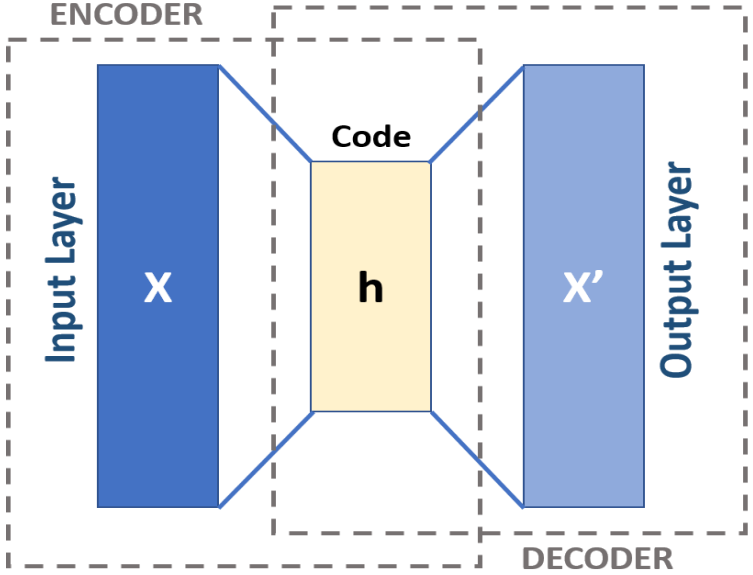
ML Approaches:

- Statistical Methods: Using Z-scores or moving averages for simple thresholding.
- Unsupervised Learning: Since "normal" data is plentiful but "anomalous" data is rare,

Unsupervised learning

- Isolation Forests: Specifically designed to isolate anomalies rather than profiling normal points.

- Autoencoders : Training a neural network to reconstruct input data. High reconstruction error indicates an anomaly.



Predictive Maintenance

- PdM moves away from "Run-to-Failure" or "Fixed-Interval" maintenance toward a data-driven approach.
- The Objective: Predicting the Remaining Useful Life (RUL) of an asset.
- Modeling Techniques:
- Regression Models: Predicting a continuous value (e.g., "This engine has 40 flight cycles left").
- Classification: Predicting if a failure will occur within a specific window (e.g., "Will this pump fail in the next 7 days?").

Feature Engineering

- This is critical in IoT. We create features like rolling means, lag variables (what happened 10 minutes ago?)

Pattern Recognition & Time-Series Analysis

- Temporal Patterns:
- Seasonality: Recognizing daily, weekly, or seasonal cycles (e.g., energy consumption patterns in a smart city).
- Trend Analysis: Identifying long-term increases or decreases in sensor values.

Pattern Recognition & Time-Series Analysis

- Advanced Architectures for Patterns:

- Long Short-Term Memory (LSTM): A type of RNN capable of learning long-term dependencies in time-series data.

- Transformers: Increasingly used in IoT for their ability to handle "attention"—focusing on specific past events that correlate with current sensor readings.

Summary

- Data is an Asset: IoT is a pipeline where raw signals are refined into economic value.
- Context is King: A temperature of 100°C is an anomaly in a fridge, but normal in a boiler. ML provides this context.
- Proactive over Reactive: The transition from detecting current faults (Anomaly Detection) to forecasting future ones (Predictive Maintenance) is the hallmark of a mature IoT system.

Practice code autoencoder

- Edge Computing: Because the Encoder portion of the model is mathematically lightweight (just matrix multiplications), you can often deploy the trained encoder directly onto IoT gateway devices or high-end microcontrollers using TensorFlow Lite.
- Dimensionality Reduction: IoT devices often suffer from "chattiness." Autoencoders help by reducing the number of features sent over the network, saving bandwidth and battery life.

Practice code autoencoder

- Thresholding: In a real-world scenario, you would define a threshold (e.g., 3 standard deviations from the mean training error). If the reconstruction error MSE exceeds this threshold, an alert is triggered.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

the "normal" state of a machine or environment is consistent. The Autoencoder learns this "normal" fingerprint. If a sensor reports something bizarre (an anomaly), the model won't know how to reconstruct it properly, causing the error rate to spike and triggering an alert.

Data Simulation and Preprocessing

- The first step mimics real-world IoT sensors like thermometers and hygrometers (measuring humidity air, gas).
- generate_iot_data: Creates a synthetic dataset of temperature, humidity, pressure, and vibration. Most of the data follows a "Normal Distribution," representing a healthy machine.

- StandardScaler: This is a critical step in Deep Learning. Neural networks struggle when one feature (like Pressure at 1013) has a much larger scale than another (like Vibration at 0.05). This scales all data so the mean is 0 and the variance is 1.

The Autoencoder Architecture

- The Encoder: It takes the 4 sensor inputs and squeezes them through a "Bottleneck" (the encoding_dim = 2 layer). This forces the network to learn only the most important patterns and ignore the noise.
- The Decoder: It takes those 2 compressed variables and tries to rebuild the original 4 sensor readings.
- Loss Function (mse): We use Mean Squared Error. The goal of training is to make the output as close to the input as possible.

Anomaly Detection Logic

- detect_anomaly: When new data arrives, the model tries to reconstruct it.

Usefulness

- Unsupervised: You don't need to manually label thousands of hours of data as "broken" or "fixed." The model learns what "fixed" looks like on its own.

- Efficiency: Once trained, the model is very small. You can run on an Edge Gateway (a small computer near the sensors) rather than sending all that data to the expensive Cloud.

Todo

- modify the code to detect a "Slow Drift"—a sensor that is slowly failing by increasing its reading by just a few degrees over time.

- 1.Calculate the reconstruction_error for the entire training set.Find the Mean and Standard Deviation sigma of these errors.

- 2.Create a new test sample where the temperature is 35°C (instead of 85°C).
- Run this through your detect_anomaly function.
- 3.Visualize how the Autoencoder "sees" the difference between normal and anomalous data.

Reference

- https://en.wikipedia.org/wiki/Autoencoder
- https://cs.stanford.edu/~quocle/tutorial2.pdf

Question

Thank you